aivancity

La Grande Ecole de l'IA & de la Data

Paris Île-de-France · Nice Côte d'Azur

Deep Learning

Introduction to Linear Regression & Gradient Descent

Burak Civitcioglu

July 12, 2026

aivancity School of AI & Data for Business & Society

Learning Goals

By the end of this lecture you should be able to:

- Understand what **machine learning** is and how it differs from traditional programming
- Define a **hypothesis function** for linear regression
- Understand the **cost function** and why we need it
- Apply **gradient descent** to find optimal parameters
- Implement simple linear regression from scratch

What is Machine Learning?

Traditional Programming:

$$\text{Data} + \text{Rules} \longrightarrow \text{Output}$$

Machine Learning:

$$\text{Data} + \text{Output} \longrightarrow \text{Rules}$$

Key insight: Instead of writing explicit rules, we **learn** them from data.

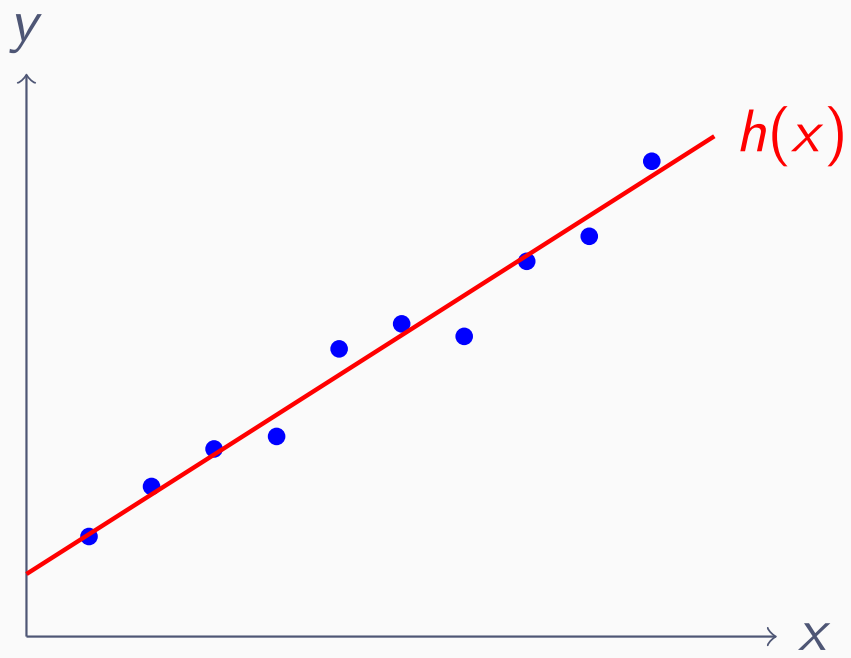
Example:

- Traditional: "If temperature > 30, turn on AC"
- ML: Learn the relationship between temperature and comfort from data

The Simplest Model: Linear Regression

Goal: Predict a continuous output y from an input x.

Assumption: The relationship is approximately linear.



Question: How do we find the "best" line?

The Hypothesis Function

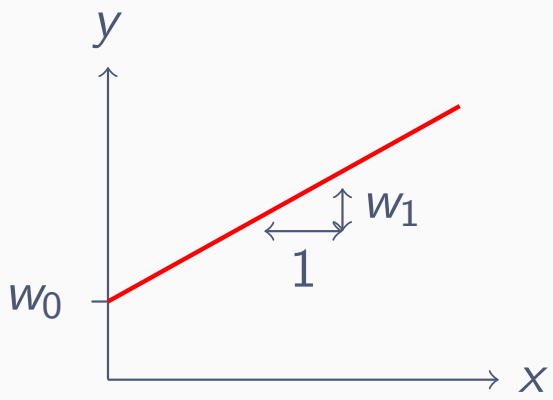
Our model (hypothesis) for predicting y from x:

$$h(x) = w_0 + w_1 \cdot x$$

Parameters:

- w_0: **bias** (y-intercept) — where the line crosses the y-axis
- w_1: **weight** (slope) — how much y changes when x increases by 1

Our goal: Find w_0 and w_1 that make $h(x)$ as close to y as possible.



How Wrong Are We? The Cost Function

Prediction error for one sample:

$$\text{error}^{(i)} = h(x^{(i)}) - y^{(i)}$$

Mean Squared Error (MSE) — average squared error over all samples:

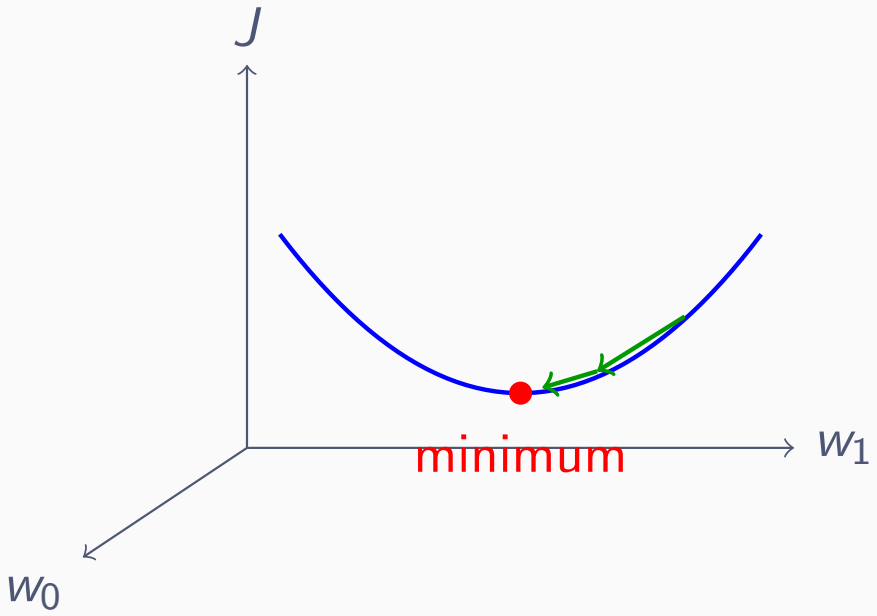
$$J(w_0, w_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

Why squared?

- Makes all errors positive (no cancellation)
- Penalizes large errors more heavily
- Mathematically convenient (smooth, differentiable)

Goal: Find w_0, w_1 that **minimize** J.

Visualizing the Cost Function



The cost function is a "bowl":

- High cost far from optimal weights
- Low cost near optimal weights
- One global minimum (for linear regression)

Gradient Descent: The Key Idea

Problem: How do we find the minimum of J?

Solution: Gradient descent — take steps downhill!

Intuition: Imagine you're blindfolded on a hilly terrain, trying to find the lowest point. Strategy: feel which way is downhill, take a step, repeat.

The gradient ∇J tells us:

- **Direction:** Which way is "uphill"
- **Magnitude:** How steep the slope is

We go opposite to the gradient (downhill)!

The Gradient Descent Update Rule

Update rule (repeat until convergence):

$$w := w - \alpha \cdot \frac{\partial J}{\partial w}$$

Components:

- w: current weight value
- α: **learning rate** — how big a step we take
- $\frac{\partial J}{\partial w}$: **gradient** — direction of steepest increase
- $-$: we go **opposite** to gradient (downhill)

For linear regression with two parameters:

$$w_0 := w_0 - \alpha \cdot \frac{\partial J}{\partial w_0}$$

$$w_1 := w_1 - \alpha \cdot \frac{\partial J}{\partial w_1}$$

Computing the Gradients

Cost function:

$$J(w_0, w_1) = \frac{1}{m} \sum_{i=1}^{m} \left(w_0 + w_1 x^{(i)} - y^{(i)} \right)^2$$

Gradient with respect to w_0:

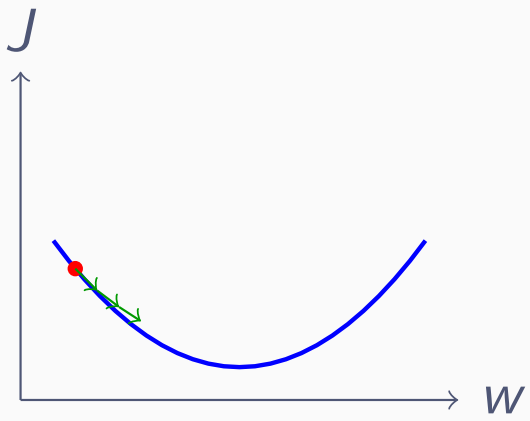
$$\frac{\partial J}{\partial w_0} = \frac{2}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right)$$

Gradient with respect to w_1:

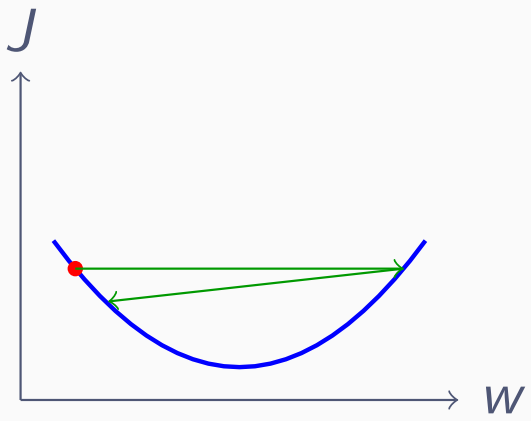
$$\frac{\partial J}{\partial w_1} = \frac{2}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)}$$

Note: The gradient is the **average error**, weighted by input for w_1.

Learning Rate: Too Big vs Too Small



Too small: slow



Too big: oscillates

Choosing learning rate α:

- Too small $\Rightarrow$ Very slow convergence
- Too large $\Rightarrow$ Overshoots, may diverge
- Just right $\Rightarrow$ Fast, stable convergence

Typical values: Start with $\alpha = 0.01$ or $\alpha = 0.1$

The Complete Algorithm

Gradient Descent for Linear Regression

1. **Initialize:** Set $w_0 = 0$, $w_1 = 0$ (or small random values)
2. **Repeat** for many iterations:
   - 2.1 Compute predictions: $h(x^{(i)}) = w_0 + w_1 x^{(i)}$
   - 2.2 Compute gradients:

$$\frac{\partial J}{\partial w_0} = \frac{1}{m}\sum_i (h(x^{(i)}) - y^{(i)}), \quad \frac{\partial J}{\partial w_1} = \frac{1}{m}\sum_i (h(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

   - 2.3 Update: $w_0 := w_0 - \alpha \frac{\partial J}{\partial w_0}, \quad w_1 := w_1 - \alpha \frac{\partial J}{\partial w_1}$
3. **Return:** Final w_0, w_1

Python Implementation

Hypothesis:

```
def h(x, w0, w1):  return w0 + w1 * x
```

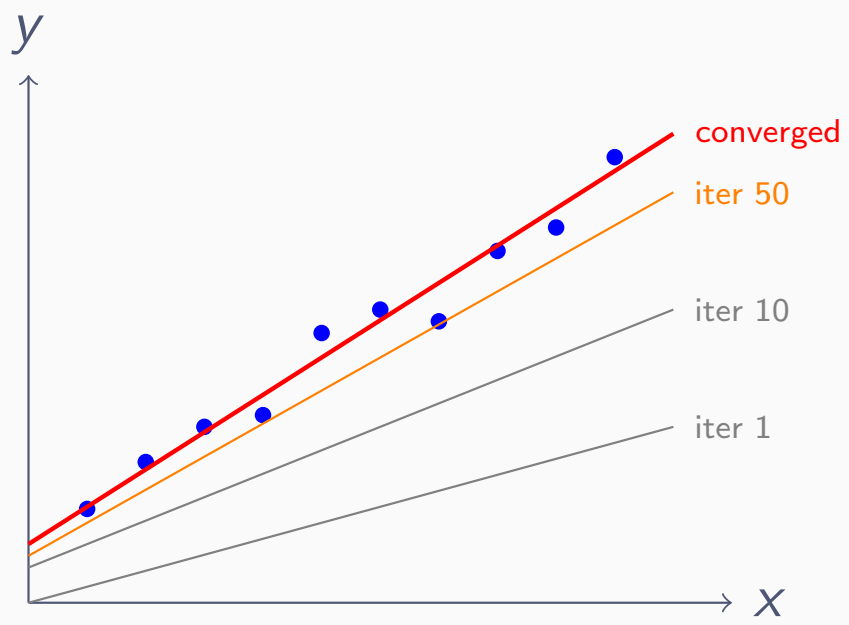
Cost function:

```
def cost(x, y, w0, w1):
return np.mean((h(x, w0, w1) - y) ** 2)
```

Gradient descent step:

```
error = h(x, w0, w1) - y
  dw0 = np.mean(error)
dw1 = np.mean(error * x)
   w0 = w0 - lr * dw0
   w1 = w1 - lr * dw1
```

Watching Gradient Descent Converge



As training progresses:

- The line gradually fits the data better
- Cost decreases each iteration
- Eventually converges to optimal solution

Why This Matters for Deep Learning

Linear regression is the building block of neural networks!

Concept	**Linear Reg.**	**Neural Networks**
Model	$h = w_0 + w_1 x$	Layers of linear + nonlinear
Parameters	w_0, w_1	Millions of weights
Cost function	MSE	Cross-entropy, etc.
Optimization	Gradient descent	Same!

Key insight: The **same algorithm** (gradient descent) trains both simple linear regression and complex deep neural networks!

Summary

- **Linear regression:** $h(x) = w_0 + w_1 x$
- **Cost function:** MSE measures how wrong we are

$$J = \frac{1}{m} \sum_i (h(x^{(i)}) - y^{(i)})^2$$

- **Gradient descent:** Iteratively update weights

$$w := w - \alpha \cdot \frac{\partial J}{\partial w}$$

- **Learning rate α:** Controls step size
- **Foundation:** Same principles apply to deep learning!

What's Next

Next lecture:

- Extend to **multiple features** (multivariate regression)
- Learn **vectorization** for efficient computation
- Understand **matrix operations** — essential for deep learning

Now: Let's implement this in the notebook!

Thank You

Let's code our first machine learning model!